Guía para entender y manejar este proyecto

HTML, CSS, JavaScript, React, Tailwind y Three.js explicados con el código real del portfolio. Pensada para leerse e imprimirse: unas 15-20 páginas.

---

0. Cómo usar esta guía

Objetivo: al terminar, poder decir "manejo estas tecnologías y sé cómo funciona el proyecto por dentro".

Lo que NO tienes que hacer:

- No hace falta memorizar nada. Los programadores consultan la documentación todo el día. Lo que importa es entender las ideas y saber dónde mirar.
- No hace falta entender todo a la primera. Cada capítulo marca lo **esencial** y lo que puedes dejar **para después**.
- No hace falta leerlo de un tirón. Son 5 sesiones de una hora.

Plan de lectura:

Sesión	Capítulos	Al acabar sabrás...
1	1, 2 y 3	Qué hace el proyecto, y HTML y CSS
2	4	JavaScript moderno
3	5 y 6	React y Tailwind
4	7	Three.js y React Three Fiber
5	8, 9 y 10	Cómo funciona todo y cómo tocarlo

Al final (capítulo 11) hay una **autoevaluación**: si puedes responderla con tus palabras, lo has entendido de verdad.

Una nota sobre sentirse saturado: es normal. Casi nadie "entiende todo" de un proyecto con 3D, scroll y despliegue. Un profesional conoce bien *su* parte y sabe moverse por el resto. Tu meta no es saberlo todo, es poder seguir el hilo y cambiar cosas sin miedo. Si un apartado se te atraganta, sáltalo, sigue y vuelve más tarde.

---

1. El proyecto en 5 minutos

Es el portfolio de Jorge Vega: **una sola página** con cuatro secciones (Home, Projects, About, Contact). Detrás del texto hay una **escena 3D**: una habitación con un avatar que se mueve según haces scroll.

La idea clave son **dos mundos que no se conocen** y comparten un tablón:

```
MUNDO HTML  (React + Tailwind)       capa de delante
texto, tarjetas, formulario, navbar
```

```
          |   escribe
          v
  TABLON: scrollState y mouseState     (objetos JavaScript normales)
          ^   lee 60 veces por segundo
          |
  MUNDO 3D  (Three.js)                 capa de detrás
  habitación, avatar, cámara
```

- Al hacer scroll, un listener escribe en el tablón (`scrollState.index`).
- La escena 3D lo lee en cada fotograma y mueve cámara y avatar.
- HTML y 3D nunca se llaman entre sí: por eso puedes tocar uno sin romper el otro.

Carpetas esenciales (`src/`):

```
sections/   el HTML de cada sección (Home, Works, About, Contact)
components/ piezas reutilizables (Navbar, ProjectCard, ProjectDialog)
scene/      todo lo que va dentro del 3D
state/      el tablón (scrollState, mouseState)
hooks/      lógica reutilizable (scroll, imán, URL...)
config/     números ajustables (cámara, coreografía, colores)
data/       los datos de los proyectos
lib/        funciones auxiliares
```

---

2. HTML: la estructura

Esencial. Es lo que ve el navegador al final de todo.

HTML describe **qué es cada cosa** con etiquetas. Un documento mínimo:

```html
<!doctype html>
<html lang="en">
  <head>
    <title>Jorge Vega</title>
    <meta name="description" content="Portfolio..." />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Este es el `index.html` del proyecto: casi vacío. React rellena `#root`.

Etiquetas que debes conocer:

Etiqueta	Para qué
`h1`…`h6`	Títulos (uno solo `h1` por página)
`p`, `a`, `ul`/`li`	Párrafo, enlace, lista
`button`	Algo que se pulsa (una acción)
`img`	Imagen (con `alt`)
`form`, `label`, `input`, `textarea`	Formulario
`header`, `nav`, `main`, `section`, `footer`	Estructura con significado
`div`, `span`	Contenedores sin significado

Semántica y accesibilidad (importante):

- Usa la etiqueta que *significa* lo que es: un `button` para acciones y un `a` para ir a otro sitio. Así el teclado y los lectores de pantalla funcionan sin esfuerzo.
- Todo `input` necesita su `label` (`for` apunta al `id` del input).
- Las imágenes llevan `alt`; si son decorativas, `alt=""`.
- Atributos `aria-*` solo cuando HTML no basta (`aria-label` en un botón que solo tiene un icono).

Dónde se ve en el proyecto: `index.html` (SEO), `sections/Contact.jsx` (formulario con `label`, `input`, `textarea`) y `App.jsx` (`<main>`).

> **Comprueba:** ¿por qué un `<div onClick>` es peor que un `<button>`?

---

3. CSS: el aspecto

Esencial.

CSS dice **cómo se ve** cada elemento: `selector { propiedad: valor; }`.

```
.card {
  padding: 1rem;
  border-radius: 1rem;
  background: var(--color-paper);
}
.card:hover { transform: translateY(-4px); }
```

3.1 La caja

Todo elemento es una caja: **contenido + padding + borde + margin**. `box-sizing: border-box` (ya viene así con Tailwind) hace que `width` incluya el padding y el borde.

3.2 Display: flex y grid

```
.fila    { display: flex; align-items: center; gap: 1rem; }
.rejilla { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; }
```

- **Flex:** una dimensión (fila o columna). Alinea y reparte espacio.
- **Grid:** dos dimensiones (filas y columnas).
- Un hijo de grid/flex no se encoge por debajo de su contenido salvo que le pongas `min-width: 0`. Es un fallo muy típico.

3.3 Posición

Valor	Efecto
`static`	Flujo normal (por defecto)
`relative`	Normal, pero sirve de referencia a sus hijos absolutos
`absolute`	Fuera del flujo, respecto al ancestro `relative`
`fixed`	Fijo a la pantalla (el canvas 3D y el navbar)
`sticky`	Normal hasta que llega a un borde y se pega

3.4 Unidades

- `px`: píxeles. `rem`: relativo al tamaño de letra base (1rem = 16px).
- `%`: relativo al padre. `vw`/`vh`: relativo a la ventana.
- `svh`: altura visible real en móviles (con la barra del navegador). Por eso el proyecto usa `svh` y no `vh`.

3.5 Variables CSS

```
:root { --color-ink: #1c1b1a; --page-fg: var(--color-ink); }
p     { color: var(--page-fg); }
```

El tema oscuro de Contact es solo **redefinir esas variables**; el resto del CSS no cambia. Está en `src/index.css`.

3.6 Responsive: media queries

```
.titulo { font-size: 2rem; }
@media (min-width: 640px) {
  .titulo { font-size: 3rem; }   /* desde 640px de ancho */
}
```

Se diseña primero para móvil y se amplía hacia pantallas grandes.

3.7 Cascada y especificidad

Cuando dos reglas chocan, gana la más **específica** y, a igualdad, la que aparece **después**. Por eso las variantes propias del proyecto (`short:`, `desk:`) se declaran al final.

Dónde se ve en el proyecto: `src/index.css` (variables, fondo animado con `@keyframes`, `@font-face`).

> **Comprueba:** ¿qué diferencia hay entre `position: absolute` y `fixed`?

---

4. JavaScript: la lógica

Esencial. React y Three.js son "solo" JavaScript.

4.1 Variables y tipos

```
const nombre = 'Ada'          // no se reasigna (usa const por defecto)
let contador = 0              // se puede reasignar
const lista = [1, 2, 3]       // array
const persona = { name: 'Ada', age: 36 }   // objeto
```

Tipos: texto (`string`), número, booleano, `null`, `undefined`, array, objeto, función. **Falsy** (se comporta como falso): `false`, `0`, `''`, `null`, `undefined`.

4.2 Funciones

```
function sumar(a, b) { return a + b }
const sumar2 = (a, b) => a + b          // función flecha
const doble = (n) => n * 2
```

Las funciones son valores: se pasan a otras funciones (`onClick={() => ...}`).

4.3 Arrays: `map`, `filter`, `sort`

Lo más usado en React. Ninguno modifica el array original.

```
const nums = [1, 2, 3, 4]
nums.map((n) => n * 2)           // [2, 4, 6, 8]   transforma cada uno
nums.filter((n) => n > 2)        // [3, 4]         se queda con algunos
nums.some((n) => n > 3)          // true           ¿alguno cumple?
[...nums].sort((a, b) => b - a)  // [4, 3, 2, 1]   (copia y ordena)
```

En el proyecto (`lib/filterProjects.js`):

```
projects
  .filter((p) => tags.length === 0 || p.tags.some((t) => tags.includes(t)))
  .filter((p) => !year || p.date.startsWith(year))
```

4.4 Objetos, desestructuración y spread

```
const { name, age = 18 } = persona     // saca propiedades (age: por defecto)
const copia = { ...persona, age: 37 }  // copia cambiando una propiedad
const mas = [...lista, 4]              // copia añadiendo
persona.address?.street                // ?. evita error si address no existe
const x = valor ?? 'defecto'           // ?? solo si valor es null/undefined
```

Spread y desestructuración están en todo el código de React.

4.5 Plantillas de texto

```
const url = `${import.meta.env.BASE_URL}models/room.glb`   // con backticks
```

4.6 Módulos

```
export function pageCount(total, perPage) { ... }   // en lib/paginate.js
import { pageCount } from '../lib/paginate'          // en otro archivo
export default ProjectCard                           // uno por defecto
```

4.7 Asincronía: `async`, `await`, `fetch`

Algunas cosas tardan (pedir datos). Una **promesa** representa algo que llegará después. `await` espera a que llegue:

```
async function enviar(datos) {
  try {
    const r = await fetch('https://api.web3forms.com/submit', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(datos),
    })
    const json = await r.json()
    return json.success
  } catch {
    return false          // sin red, etc.
  }
}
```

Es la esencia de `lib/sendContact.js`.

4.8 El DOM y los eventos

El navegador ofrece el árbol de la página como objetos:

```
const el = document.getElementById('projects')
el.getBoundingClientRect().top      // posición en pantalla
window.addEventListener('scroll', alHacerScroll, { passive: true })
window.removeEventListener('scroll', alHacerScroll)
```

Los eventos (`click`, `scroll`, `pointermove`, `keydown`) avisan de lo que hace el usuario. **Todo listener que añades hay que quitarlo** cuando ya no sirve.

4.9 Tiempo

- `setTimeout(fn, ms)`: ejecuta una vez más tarde.
- `requestAnimationFrame(fn)`: ejecuta en el siguiente fotograma (~60/s). Es la base de toda animación.

> **Comprueba:** ¿qué devuelve `[1,2,3].map((n) => n * 10)` y cambia el array original?

---

5. React: la interfaz

Esencial. Es el cuerpo del proyecto.

React te deja describir la interfaz como **funciones que devuelven HTML**. Cuando los datos cambian, vuelve a ejecutar la función y actualiza solo lo necesario.

5.1 Componentes y JSX

```
export default function Footer() {
  return <footer><p>© {new Date().getFullYear()} Jorge Vega</p></footer>
}
```

- Nombre en mayúscula; devuelve **un** elemento raíz (o `<>...</>`).
- Entre `{ }` va cualquier expresión JavaScript.
- Se escribe `className` y `htmlFor` (no `class`/`for`).
- Se usan como etiquetas: `<Footer />`. `App.jsx` es un árbol de componentes.

5.2 Props

Argumentos de un componente, de padre a hijo. Nunca se modifican dentro.

```
function ProjectCard({ title, tags = [], compact = false }) { ... }
```

```
<ProjectCard {...project} compact={compact} />   // {...} esparce el objeto
```

5.3 Listas, `key` y condicionales

```
{visible.map((p) => <ProjectCard key={p.id} {...p} />)}   // key única
{href && <a href={href}>Visit website</a>}                 // si hay href
{comingSoon ? title : <button>{title}</button>}           // una u otra
```

5.4 Estado: `useState`

```
const [status, setStatus] = useState('idle')   // 'idle'|'sending'|...
const [values, setValues] = useState({ name: '', email: '' })

setValues((current) => ({ ...current, name: 'Ada' }))   // copia, nunca muta
```

Reglas: no mutar el estado; actualizar crea un valor nuevo; si depende del anterior, usar la forma `(prev) => ...`. Al cambiar, React vuelve a ejecutar el componente.

5.5 Efectos: `useEffect`

Código que corre **después** de pintar: listeners, mediciones, timers.

```
useEffect(() => {
  const onMove = (e) => { mouseState.x = (e.clientX / innerWidth) * 2 - 1 }
  window.addEventListener('pointermove', onMove)
  return () => window.removeEventListener('pointermove', onMove)  // limpieza
}, [])    // [] = solo al montar; [a, b] = cuando cambien a o b
```

En desarrollo, **StrictMode** ejecuta los efectos dos veces para destapar limpiezas que faltan.

5.6 Refs: `useRef`

Caja mutable que **no provoca renders**. Dos usos: agarrar un elemento del DOM (`ref={closeRef}` + `closeRef.current.focus()`) y guardar un valor "de fondo".

5.7 `useMemo`, `useCallback` y hooks propios

- `useMemo` guarda un cálculo; `useCallback` guarda una función. No los uses "por si acaso".
- Un **hook propio** es una función `useAlgo` que usa otros hooks. En el proyecto: `useScrollProgress`, `useScrollMagnet`, `useSectionUrl`, `useAfterFirstPaint`... Encapsulan lógica y mantienen limpios los componentes.

5.8 Cosas que verás y conviene reconocer

Cosa	Para qué sirve	Dónde
`lazy` + `Suspense`	Cargar un trozo de código solo cuando hace falta	`App.jsx` (el 3D)
`createPortal`	Dibujar fuera de su sitio (ventanas modales)	`ProjectDialog.jsx`
Error boundary	Si algo falla, que la web siga	`SceneErrorBoundary.jsx`
`useSyncExternalStore`	Leer algo del navegador (media queries)	`useMedia` en `Works.jsx`

> **Comprueba:** ¿por qué un efecto que añade un listener debe devolver una función de limpieza?

---

6. Tailwind CSS: estilos con clases

Esencial. Es CSS escrito de otra forma, no CSS nuevo.

En vez de escribir una clase con muchos estilos en otro archivo, usas muchas clases pequeñas (cada una hace **una cosa**) directamente en el JSX:

```
<button className="inline-flex h-11 items-center rounded-full px-6 text-base">
```

Clase	Equivale a
`inline-flex`	`display: inline-flex`
`h-11`	`height: 2.75rem` (11 x 0,25 rem)
`items-center`	`align-items: center`
`rounded-full`	esquinas totalmente redondas
`px-6`	`padding-left/right: 1.5rem`

La escala: el número x 0,25 rem. `p-4` = 1 rem = 16 px.

6.1 Prefijos: responsive y estados

```
<h2 className="text-4xl sm:text-5xl">           // 5xl desde 640 px
<a className="hover:underline focus-visible:outline-2">
<button className="disabled:opacity-30">
<div className="max-sm:hidden">                 // solo por debajo de 640 px
```

Anchos: `sm` 640, `md` 768, `lg` 1024, `xl` 1280. Sin prefijo vale para todos (primero móvil, luego se amplía).

El proyecto define dos variantes propias en `index.css`: **`short:`** (pantallas de 600 px de alto o menos) y **`desk:`** (1200 px o más).

6.2 Valores y variables

```
className="min-h-[14.5rem] bg-(--color-paper)/85 max-w-[min(17rem,24svh)]"
```

- Corchetes `[...]` para valores que no están en la escala.
- Paréntesis `(--variable)` para variables CSS, y `/85` para opacidad.

6.3 Trucos que verás

Clase	Qué hace
`sr-only`	Texto solo para lectores de pantalla
`invisible` / `hidden`	Oculta conservando / quitando su hueco
`min-w-0`	Deja que un hijo flex/grid se encoja
`touch-pan-y`	El dedo solo hace scroll vertical ahí
`backdrop-blur-md`	Desenfoca lo que hay detrás (cristal)

Una trampa real del proyecto: una fila de chips sin saltos de línea dentro de una columna de grid la ensanchó más que la pantalla. Solución: `grid-cols-[minmax(0,1fr)]` y `min-w-0`.

> **Comprueba:** ¿qué diferencia hay entre `hidden` e `invisible`?

---

7. Three.js y React Three Fiber: el 3D

Esencial lo de 7.1 a 7.4; el resto puedes dejarlo para después.

7.1 Los ingredientes de una escena

Ingrediente	Analogía
Escena	El plató
Mesh = geometría + material	Los objetos (forma + aspecto)
Luces	La iluminación
Cámara	La cámara de cine
Renderer	"Fotografía" la escena y la pinta en un `<canvas>`

Y un **bucle**: ~60 veces por segundo se actualiza la escena y se vuelve a "fotografiar". Cada imagen es un *frame*.

7.2 Transformaciones y árbol

Todo objeto tiene `position [x, y, z]`, `rotation` (en **radianes**) y `scale`. Ejes: x derecha, y arriba, z hacia ti.

Los objetos forman un **árbol**: un hijo hereda la transformación del padre. Si giras la habitación, el avatar (su hijo) gira con ella (`scene/Room.jsx`).

7.3 Modelos y animaciones

- **GLB/glTF:** formato estándar de modelos 3D (`room.glb`, `avatar.glb`).
- **Huesos:** el avatar tiene un esqueleto; una animación es "cómo gira cada hueso en el tiempo".
- **Clip, action, mixer:** el clip son los datos, la *action* es "reproducirlo" y el *mixer* mezcla (*crossfade* entre "sentado" y "cayendo").
- Las animaciones vienen de Mixamo en FBX y el proyecto las convierte a JSON (`scripts/convert-animations.mjs`) para que pesen 10 veces menos.

7.4 Delta y suavizado

`delta` = segundos desde el frame anterior. Mover algo `velocidad * delta` hace que vaya igual en un PC lento y en uno rápido. `MathUtils.damp(actual, objetivo, lambda, delta)` acerca un valor a su objetivo **suavemente**; así la cámara no da tirones:

```
c.zoom = MathUtils.damp(c.zoom, goal.zoom, 5, delta)   // CameraRig.jsx
```

7.5 React Three Fiber (R3F)

Escribes la escena en JSX; cada etiqueta en minúscula es un objeto de three:

```
<Canvas camera={{ position: [5, 3.5, 6], fov: 40 }}>
  <ambientLight intensity={0.5} />
  <mesh position={[0, 1, 0]}>
    <boxGeometry args={[1, 1, 1]} />
    <meshStandardMaterial color="#f2a6d8" />
  </mesh>
</Canvas>
```

Pieza	Para qué
`<Canvas>`	Crea escena, cámara, renderer y bucle
`useFrame((state, delta) => ...)`	Código que corre **cada frame**
`<primitive object={x} />`	Inserta un objeto three ya creado (un modelo)
`useGLTF(url)` (drei)	Carga un `.glb` y suspende hasta que llega
`useAnimations` (drei)	Crea el mixer y las acciones

Regla de oro: dentro de `useFrame` **nunca** llames a `setState` (React redibujaría 60 veces por segundo). Se mutan objetos y refs directamente.

> **Comprueba:** ¿por qué el avatar se mueve con la habitación cuando esta gira? ¿Y por qué se usa `damp` en lugar de asignar el valor directo?

---

8. Cómo funciona el proyecto por dentro

8.1 `App.jsx`: el director

```
export function App() {
  useSectionSpacing()   // huecos entre secciones
  useScrollProgress()   // rellena scrollState al hacer scroll
  useScrollMagnet()     // imán de scroll
  useSectionUrl()       // URL <-> sección
  const startScene = useAfterFirstPaint()   // ¿cargamos ya el 3D?
  return (
    <>
      <div className="backdrop" />                {/* fondo */}
      {startScene && <Suspense><Experience /></Suspense>}   {/* 3D */}
      <Navbar />
      <main><Home /><Works /><About /><Contact /></main>
      <Footer />
    </>
  )
}
```

Las capas, de atrás a delante: fondo, canvas 3D (`fixed`, sin capturar clics), contenido (`z-10`), navbar, ventana de proyecto. El texto aparece primero y el 3D llega después sin bloquearlo (`lazy`).

8.2 El tablón y por qué no usa `useState`

```
// state/scrollState.js
export const scrollState = { progress: 0, index: 0, offset: 0, section: 'home' }
```

El scroll cambia decenas de veces por segundo. Con `useState`, toda la página se redibujaría constantemente. Con un objeto normal: el listener **escribe** sin renders y la escena **lee** en `useFrame`. Solo pasa a estado de React lo que debe cambiar la interfaz (el enlace activo del navbar).

8.3 Los huecos (slots): HTML manda, el 3D se adapta

```
<div id={WORKS_AVATAR_SLOT_ID} />      {/* "aquí va el avatar" */}
```

`CameraRig` mide ese `div` y calcula zoom y desplazamiento para que el avatar caiga dentro. Cambias el layout y el 3D lo sigue solo.

8.4 Del scroll al `index`

1. `lib/sectionGeometry.js`: `tops[i]` = scroll al que cada sección llega arriba; `rests[i]` = hasta cuándo la escena se queda quieta.
2. `useScrollProgress` convierte `scrollY` en un número continuo `index`: 0 = Home, 1 = Projects, 2 = About, 3 = Contact; `1.5` = a mitad de camino.

3. Cada tramo tiene una **ventana** (`CAMERA_WINDOWS`): la transición solo ocurre entre, por ejemplo, el 15 % y el 75 % del tramo. Fuera de ella la sección se ve **quieta** (la "meseta").
4. `ease(t)` suaviza el arranque y el final.

8.5 Tres hooks de comportamiento

- **Imán (`useScrollMagnet`):** si sueltas el scroll entre secciones, termina de colocarlo según la dirección del gesto. Suave y nunca pelea con el usuario (no actúa con el dedo o el botón del ratón pulsado).
- **Espaciado (`useSectionSpacing`):** acerca secciones con un margen negativo, con topes para que nunca se vean dos secciones mezcladas.
- **URL (`useSectionUrl`):** `history.replaceState` pone `/portfolio/projects` al llegar a esa sección, y al abrir esa URL te lleva a ella.

8.6 La escena

- **`Experience.jsx`:** `<Canvas>`, luces y `PerformanceMonitor` (baja la resolución si caen los fps).
- **`CameraRig.jsx`:** un keyframe de cámara por sección (`config/cameraKeyframes.js`); mezcla y amortigua en cada frame.
- **`Room.jsx`:** colorea la habitación por pieza (`config/roomPalette.js`), la inclina con el ratón y la desvanece con el scroll.
- **`Avatar.jsx`:** elige la animación por fase y mira a la cámara:

```
function phaseFor(index) {
  if (index < SIT_UNTIL) return 'sitting'
  if (index < LAND_AT) return 'falling'
  if (index >= CONTACT_FALL[0] && index < CONTACT_FALL[1]) return 'falling'
  return 'breathing'
}
```

El avatar **no usa opacidad** para desaparecer (es una malla abierta y se vería su interior): se oculta con `visible = false` y vuelve cayendo. Y el giro de cabeza parte siempre de la pose base del hueso para no acumularse.

8.7 Projects

```
const filtered = useMemo(() => filterProjects(PROJECTS, filters), [filters])
const pages = pageCount(filtered.length, perPage)
const visible = pageItems(filtered, page, perPage)
```

Flujo claro: **datos -> filtro -> paginado -> pintado**. `filterProjects` y `paginate` son **funciones puras** (mismas entradas, misma salida, sin efectos), fáciles de entender y de probar. El número de cards por página depende del ancho y del alto (`useMedia`). Pulsar una card abre `ProjectDialog`: un portal en `<body>` que "nace" de la card, atrapa el foco y se cierra con Escape.

8.8 Contact

Máquina de estados: `idle -> sending -> success | error`. Valida en cliente (mensajes con `aria-describedby`), envía con `fetch` a Web3Forms (`lib/sendContact.js`) e incluye un **honeypot** (campo señuelo para bots). La clave del servicio es pública por diseño. **Nunca** pongas secretos en variables `VITE_`: todo lo que lleva ese prefijo acaba en el navegador.

8.9 Rendimiento y accesibilidad en una lista

- El 3D es un chunk aparte y arranca cuando el navegador está libre.
- Modelos, animaciones y fuente se precargan; animaciones en JSON (~108 KB).
- Calidad adaptativa (resolución y sombras) y menos en táctil.
- Capturas WebP con miniaturas.
- Teclado completo, foco visible, `aria-live`, 3D `aria-hidden`, `prefers-reduced-motion`, enlace "Skip to main content".

8.10 Despliegue

1. `npm run build` genera `dist/` (la web final, minimizada).
2. `vite.config.js` fija `base: '/portfolio/'` porque la web vive en una subcarpeta. En JavaScript las rutas usan `import.meta.env.BASE_URL`.
3. `.github/workflows/deploy.yml`: en cada `push` a `main`, GitHub Actions instala, construye y publica en GitHub Pages en 1-2 minutos.
4. Las URLs por sección funcionan porque la build copia `index.html` a `404.html` (GitHub Pages solo sirve archivos que existen).
5. Variables en `.env`: `VITE_SITE_URL` y `VITE_WEB3FORMS_KEY`.

---

9. Recetas: cambios típicos

Añadir un proyecto: guarda la captura en `public/projects/` (WebP de 960 y de 480 px) y sustituye una card "Coming soon" en `src/data/projects.js`:

```
{
  id: 'mi-proyecto', title: 'Mi proyecto',
  description: 'Frase corta.', details: 'Texto largo para la ventana.',
  tags: ['React'], date: '2026-11',
  href: 'https://...', repo: 'https://github.com/...',
  image: 'projects/mi-proyecto.webp',
  thumb: 'projects/mi-proyecto-thumb.webp', accent: 'peach',
},
```

Chips, años y páginas se generan solos.

Cambiar colores: `src/index.css` (variables), `config/roomPalette.js` (habitación 3D).

Cambiar textos: `sections/*.jsx`, `data/projects.js`, `index.html`.

Ajustar el encuadre del 3D: `config/cameraKeyframes.js`, campo `frame`: `meters` (menos = más zoom), `fill` y `biasX`.

Cambiar el correo del formulario: crea otra clave en Web3Forms con ese correo y cambia `VITE_WEB3FORMS_KEY`.

Flujo diario:

```
npm run dev      # trabajar
npm run lint     # revisar
npm run build    # comprobar que construye
git add -A && git commit -m "feat: ..." && git push   # despliega solo
```

---

10. Cuando algo falla

Abre la consola del navegador (F12) y la pestaña Network: casi siempre hay una pista. Lee el mensaje entero (indica archivo y línea) y cambia **una cosa cada vez**.

Síntoma	Causa probable	Solución
404 en `/models/...`	Ruta absoluta que ignora `base`	`import.meta.env.BASE_URL`
Blanco al desplegar	`base` no coincide con el repo	Revisar `vite.config.js`
El formulario no envía	Falta o es inválida la clave	Revisar `.env` y Network
Algo ocupa más que la pantalla	Hijo de grid sin `min-w-0`	`min-w-0` / `minmax(0,1fr)`
Se duplica solo en desarrollo	StrictMode (efecto sin limpieza)	Añadir la limpieza
Acentos rotos (`Ã¡`)	Editar con PowerShell 5.1	Usar el editor, UTF-8
El avatar no se anima	Huesos con otro nombre	Comprobar `Hips`, `Head`...
404 en consola en `/home`	Truco del `404.html`	Es esperado

---

11. Autoevaluación: ¿lo manejo?

Responde con tus palabras, sin mirar. Si dudas en alguna, vuelve a ese capítulo (entre paréntesis).

Fundamentos

1. ¿Qué diferencia hay entre `div` y `section`, y entre `button` y `a`? (2)
2. ¿Qué hace `display: flex` y para qué sirve `gap`? (3)
3. ¿Qué devuelve `[1,2,3].map((n) => n * 10)`? ¿Cambia el original? (4)
4. ¿Qué hace `{ ...persona, age: 37 }`? (4)
5. ¿Qué es una promesa y qué hace `await`? (4)

React y Tailwind 6. ¿Por qué no se debe mutar el estado directamente? (5) 7. ¿Para qué sirve el array de dependencias de `useEffect`? (5) 8. ¿Qué diferencia hay entre estado y ref? (5) 9. Escribe las clases de un botón gris en móvil y negro desde 768 px. (6)

3D 10. ¿Qué es `useFrame` y por qué no se hace `setState` dentro? (7) 11. ¿Para qué sirve `delta`? ¿Y `damp`? (7)

El proyecto 12. ¿Cómo sabe el 3D cuánto scroll llevas, si HTML y 3D no se hablan? (8.2) 13. ¿Qué es el `index` y qué valor tiene entre Projects y About? (8.4) 14. ¿Qué cambiarías para que el avatar de Projects se vea más grande? (9) 15. ¿Qué pasa desde que haces `git push` hasta que se ve en internet? (8.10)

Mini-ejercicios (en una rama):

- Cambia un texto del Home y compruébalo en `npm run dev`.
- Añade un proyecto de prueba y mira cómo aparece su chip en el filtro.
- Cambia el color de `--color-peach` y observa qué elementos cambian.
- Cambia `meters` del keyframe de Projects y mira el zoom del avatar.
- Escribe una prueba para `pageCount` con `node:test` y ejecútala con `node --test`.

Si respondes 12 de 15 con soltura, **manejas el proyecto**. Con 15 de 15 podrías explicárselo a otra persona; y eso es entenderlo de verdad.

---

12. Glosario y recursos

Término	Significado
Componente	Función de React que devuelve JSX
Props	Datos que recibe un componente del padre
Estado	Memoria de un componente; al cambiar, se redibuja
Hook	Función `useAlgo` que añade capacidades a un componente
Efecto	Código que corre tras pintar (listeners, mediciones)
Ref	Caja mutable que no provoca renders
Mesh	Objeto 3D: geometría + material
Frame	Cada imagen del bucle de render
Delta	Tiempo desde el frame anterior
Damping	Suavizado hacia un objetivo
GLB	Formato estándar de modelos 3D para la web
Clip / mixer	Datos de una animación / quien los reproduce y mezcla
Build	Versión optimizada lista para publicar (`dist/`)
Chunk	Cada archivo JavaScript en que se parte la build
Lazy loading	Cargar algo solo cuando hace falta
Función pura	Mismas entradas, misma salida, sin efectos
Honeypot	Campo señuelo para cazar bots
ARIA	Atributos que describen la interfaz a lectores de pantalla

Dónde seguir (documentación oficial, siempre la mejor primera fuente):

- MDN (HTML, CSS, JavaScript): developer.mozilla.org
- React: react.dev/learn
- Tailwind: tailwindcss.com/docs
- Three.js: threejs.org/manual y threejs.org/examples
- React Three Fiber: docs.pmnd.rs/react-three-fiber
- Vite: vite.dev/guide

Orden de estudio sugerido: HTML y CSS (MDN) -> JavaScript moderno -> tutorial de react.dev -> utilidades de Tailwind -> una escena mínima de three.js (un cubo giratorio) -> la misma en R3F -> volver a esta guía.

> Última recomendación: elige una sección pequeña (por ejemplo el formulario de contacto), síguela de arriba abajo y explícasela a alguien. Cuando puedas, la has entendido.